

PRATICAL EXECUTION_CO-4

1. Virtual Memory

```
GNU nano 8.7.1 vi
#include <stdio.h>

int main()
{
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

```
Received: Hello from WileE
hemasai@Hemasai:~/OSSP_Assignment$ nano virtual_memory.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc virtual_memory.c -o virtual_memory
hemasai@Hemasai:~/OSSP_Assignment$ ./virtual_memory
Value of x = 10
Virtual address of x = 0x7ffffeb201164
hemasai@Hemasai:~/OSSP_Assignment$ |
```

2. Memory Layout

```

GNU nano 2.7.1
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main()
{
    int local = 20;

    int *heap = malloc(sizeof(int));

    *heap = 30;

    printf("Code address   : %p\n", (void *)main);
    printf("Data address   : %p\n", (void *)&global);
    printf("BSS address    : %p\n", (void *)&bss_variable);
    printf("Heap address   : %p\n", (void *)heap);
    printf("Stack address  : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

```

Virtual address of x = 0x7ffffeb201164
hemasai@Hemasai:~/OSSP_Assignment$ nano memory_layout.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc memory_layout.c -o memory_layout
hemasai@Hemasai:~/OSSP_Assignment$ ./memory_layout
Code address   : 0x62d0629c91a9
Data address   : 0x62d0629cc010
BSS address    : 0x62d0629cc018
Heap address   : 0x62d06dfc1010
Stack address  : 0x7ffffebe257cc
hemasai@Hemasai:~/OSSP_Assignment$ nano memory_layout.c
hemasai@Hemasai:~/OSSP_Assignment$ |

```

3. Demand Paging

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main()
{
    int local = 20;

    int *heap = malloc(sizeof(int));

    *heap = 30;

    printf("Code address   : %p\n", (void *)main);
    printf("Data address    : %p\n", (void *)&global);
    printf("BSS address     : %p\n", (void *)&bss_variable);
    printf("Heap address     : %p\n", (void *)heap);
    printf("Stack address    : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

```

Heap address   : 0x62d06d+c1010
Stack address  : 0x7fffebe257cc
hemasai@Hemasai:~/OSSP_Assignment$ nano memory_layout.c
hemasai@Hemasai:~/OSSP_Assignment$ nano page_table.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc page_table.c -o page_table
hemasai@Hemasai:~/OSSP_Assignment$ ./page_table
Enter page number (0-3): 1
Page is already in memory.
Physical frame = 1
hemasai@Hemasai:~/OSSP_Assignment$ nano memory_layout.c
hemasai@Hemasai:~/OSSP_Assignment$ |

```

4. Address Translation

```

GNU nano 8.7.1
#include <stdio.h>

int main()
{
    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;

    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address =
        frame * page_size + offset;

    printf("Virtual Address = %d\n",
        virtual_address);

    printf("Page Number = %d\n",
        page);

    printf("Offset = %d\n",
        offset);

    printf("Frame Number = %d\n",
        frame);

    printf("Physical Address = %d\n",
        physical_address);

    return 0;
}

```

```

Virtual Address = 250
hemasai@Hemasai:~/OSSP_Assignment$ nano memory_layout.c
hemasai@Hemasai:~/OSSP_Assignment$ nano address_translation.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc address_translation.c -o address_translation
hemasai@Hemasai:~/OSSP_Assignment$ ./address_translation
Virtual Address = 250
Page Number = 2
Offset = 50
Frame Number = 1
Physical Address = 150
hemasai@Hemasai:~/OSSP_Assignment$ nano address_translation.c
hemasai@Hemasai:~/OSSP_Assignment$ |

```

5. Allocation Memory

```

hemasai@Hemasai: ~/OSSP_/  X  +  v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

hemasai@Hemasai:~/OSSP_Assignment$ nano address_translation.c
hemasai@Hemasai:~/OSSP_Assignment$ nano allocation_memory.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc allocation_memory.c -o allocation_memory
hemasai@Hemasai:~/OSSP_Assignment$ ./allocation_memory
10 20 30 40 50
hemasai@Hemasai:~/OSSP_Assignment$

```

6. Accessing Memory

```
GNU nano 2.7.1
#include <stdio.h>

int main()
{
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

```
hemasai@Hemasai:~/OSSP_Assignment$ nano accessing_memory.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc accessing_memory.c -o accessing_memory
hemasai@Hemasai:~/OSSP_Assignment$ ./accessing_memory
Before accessing memory
Segmentation fault      ./accessing_memory
hemasai@Hemasai:~/OSSP_Assignment$ |
```